

B9 Series B

UMAP / t-SNE: parameters, randomness, reproducibility

Pin the seed, explain what each knob does to the picture — and stop being fooled by UMAP.

About 30 min | Prerequisites: B8 | Data: PBMC 3k

Same data, same line of code

your machine



your labmate's machine



same data
same line of code



cluster membership is identical — only the layout moved. Can anyone reproduce your figure?

simulated data (run for real)

Identical membership, different layout. Could you reproduce your own figure in three months?

WHERE THIS EPISODE STARTS

Leave the randomness unmanaged, and your paper ships a figure no one can reproduce.

Including future you. This episode hunts the randomness down, pins it, and writes it into the methods.

What you will learn

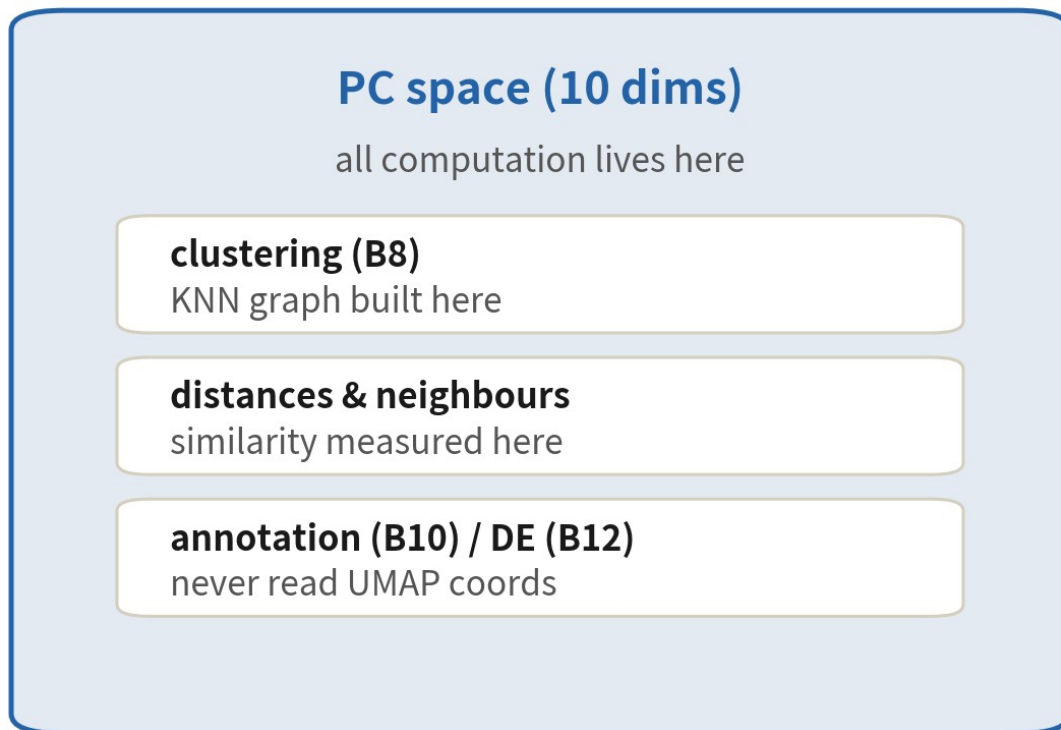
- 1 Run RunUMAP / RunTSNE and pin the result with the three levels of seeding
- 2 Sweep n.neighbors and min.dist and state what each one controls
- 3 Recite the three A5 misreads and produce a publication-grade UMAP

01

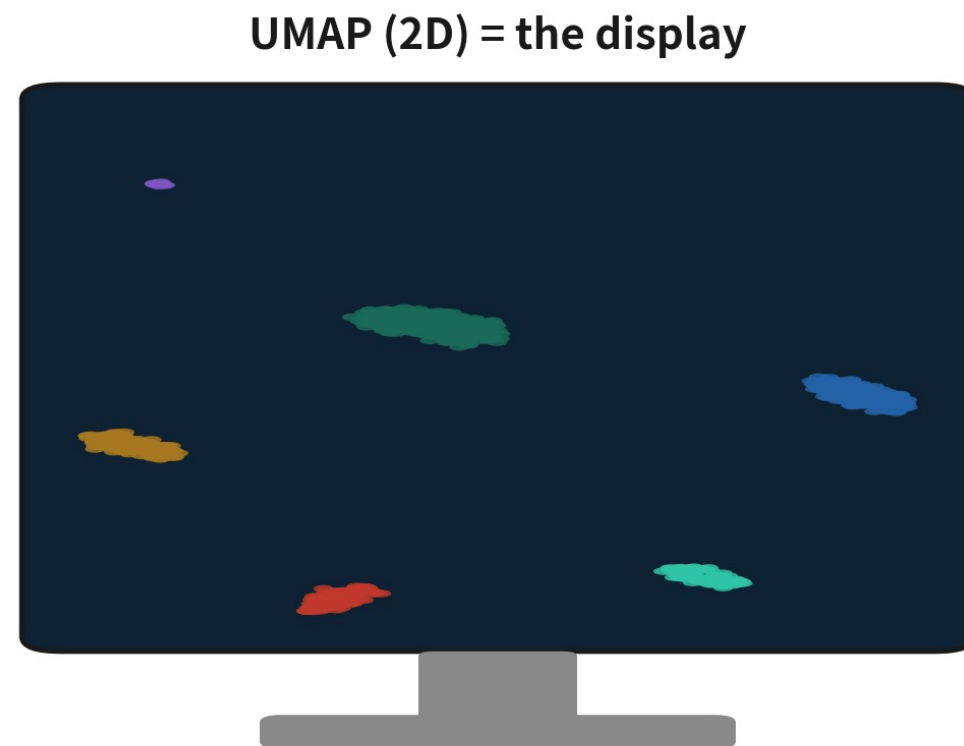
UMAP is a display, not an analysis

Put it in its place and half the parameter anxiety disappears

All the analysis happens in PC space



RunUMAP
→
**one-way street:
nothing flows back**



a projection for your eyes; its coordinates enter no statistics

Clustering, distances, annotation and DE all read PC space; UMAP coordinates only feed your eyes

Where the randomness comes from

1. initialization



each cell's 2D starting point,
random or spectral

seed controls it

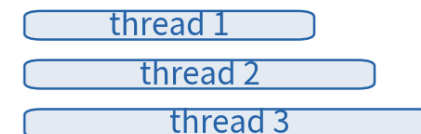
2. negative sampling



each SGD step draws random
non-neighbours to push away

seed controls it

3. parallelism



multi-threaded floating-point
addition order varies per run

seed cannot

seed pins down the first two; the third needs single-threading and version records — hence the three levels

schematic

- initialization: each cell's 2D starting point
- negative sampling: random non-neighbours pushed away each step
- parallelism: thread-order arithmetic that no seed controls
- seed pins the first two; single-threading and version records handle the third

CORE IDEA

**UMAP is a projection drawn for your eyes.
Its coordinates enter no statistics.**

So the knobs and the seed change the picture, never your conclusions — as long as you never compute on the coordinates.

02

RunUMAP and the three levels of seeding

Run it, then pin the randomness level by level

RunUMAP: one line to launch

```
pbm c <- readRDS("output/pbm c_b08_clustered.rds") # B8 的 10 群  
  
pbmc <- RunUMAP(pbm c, dims = 1:10) # dims 承接 B7 的 nPC  
DimPlot(pbm c, reduction = "umap", label = TRUE)
```

Warning: The default method for RunUMAP has changed from calling Python UMAP via reticulate to the R-native UWOT using the cosine metric

This warning is information, not an error: v5 defaults to the R-native uwot engine — no Python needed

RunUMAP's defaults, called out

dims = 1:10



not a default — your
B7 decision; match
what FindNeighbors
used

**n.neighbors = 30,
min.dist = 0.3**



the two look-and-feel
knobs; swept in the
next section

seed.use = 42



Seurat fixes the seed
for you — why your
reruns match

Rerun and verify reproducibility

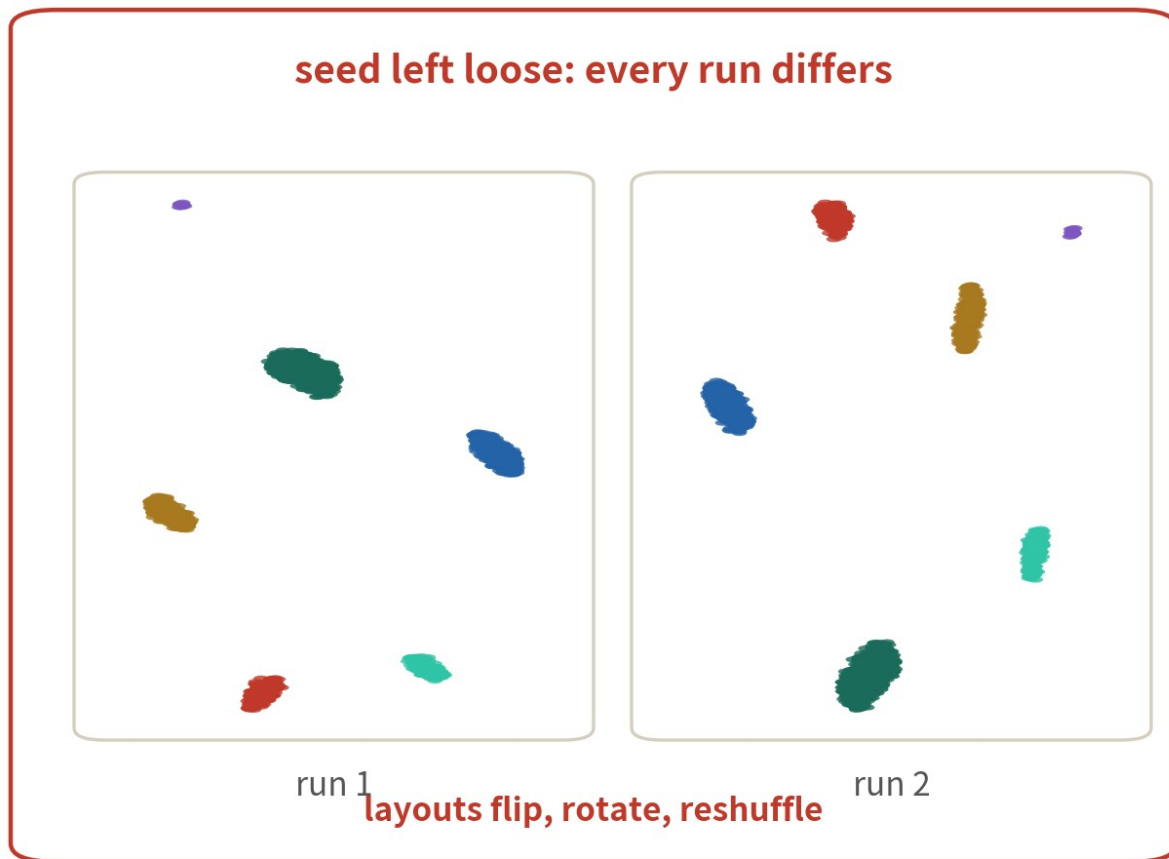
```
u1 <- Embeddings(pbm c, "um ap")
pbm c <- RunUMAP(pbm c, dim s = 1:10) # 同參數再跑一次
u2 <- Embeddings(pbm c, "um ap")
all.equal(u1, u2)

# 想看「另一種長相」：換 seed，其他都不動
pbm c <- RunUMAP(pbm c, dim s = 1:10, seed.use = 123,
  reduction.name = "um ap_alt")
```

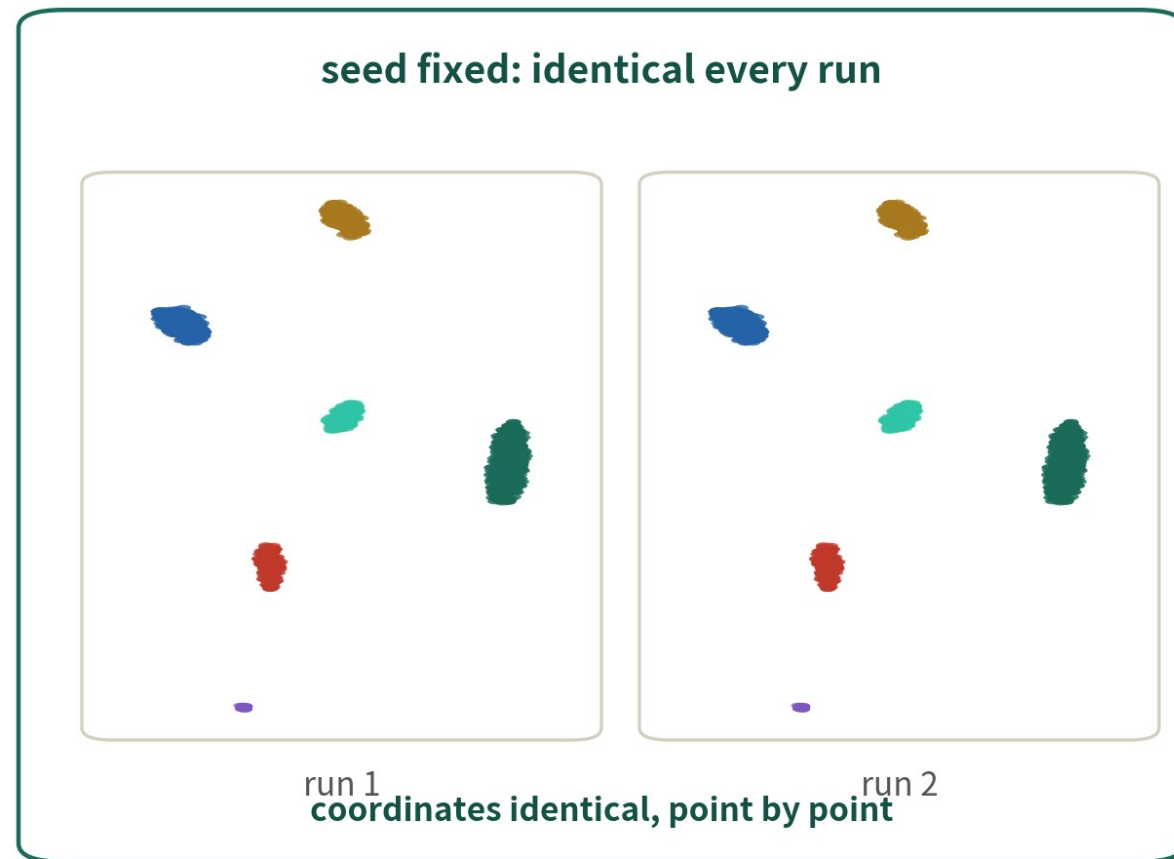
```
[1] TRUE
```

Same machine, versions and parameters → identical point by point, courtesy of seed.use = 42

Before and after pinning the seed



simulated data (run for real)



Seurat's RunUMAP defaults to seed.use = 42

Left: loose seed, a new layout every run. Right: pinned seed, identical every time

FOR YOUR METHODS SECTION

"UMAP: RunUMAP(dims = 1:10), all other arguments including seed.use = 42 at defaults; Seurat and uwot versions pinned in renv.lock."

All three levels in place: global set.seed, function-level seed.use, and implementation versions and threading. Miss one and the figure can drift.

03

Two knobs: `n.neighbors` and `min.dist`

Do not memorize definitions — sweep and watch

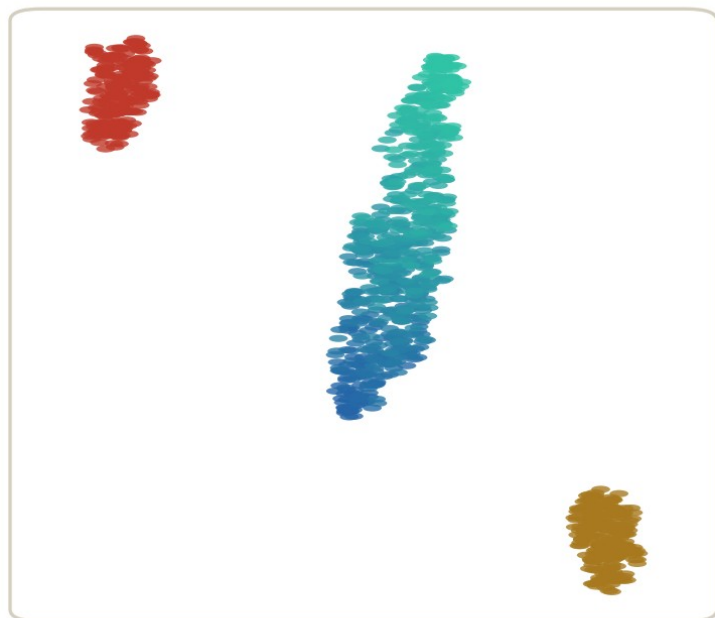
Sweep n.neighbors: 5 / 30 / 100

```
for (nn in c(5, 30, 100)) {  
  pbm c <- RunUMAP(pbm c, dim s = 1:10, n.neighbors = nn,  
    reduction.name = paste0("um ap_nn", nn),  
    reduction.key = paste0("UM APnn", nn, "_"),  
    verbose = FALSE)  
}  
p <- lapply(c(5, 30, 100), \(nn) Dim Plot(pbm c,  
  reduction = paste0("um ap_nn", nn)) + NoLegend())  
patchwork::wrap_plots(p, ncol = 3)
```

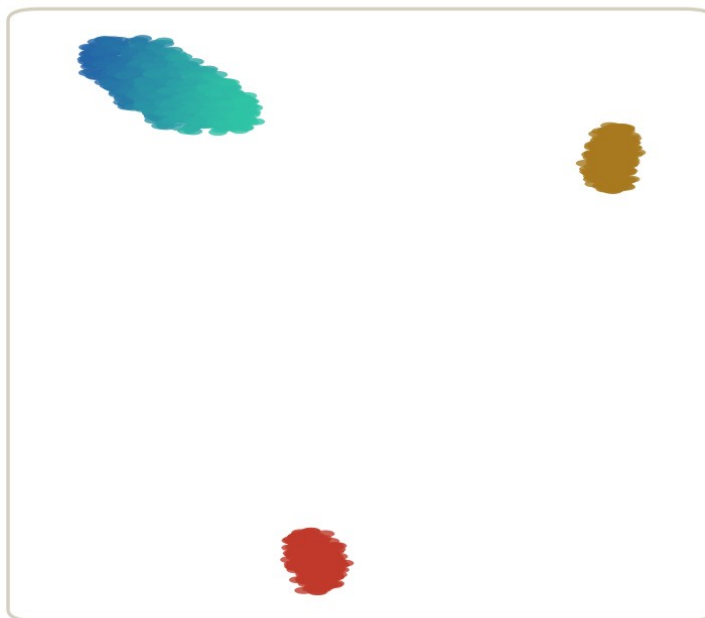
Each result lands in its own reduction, nothing overwritten — the same habit as B8's resolution sweep

n.neighbors: local vs global

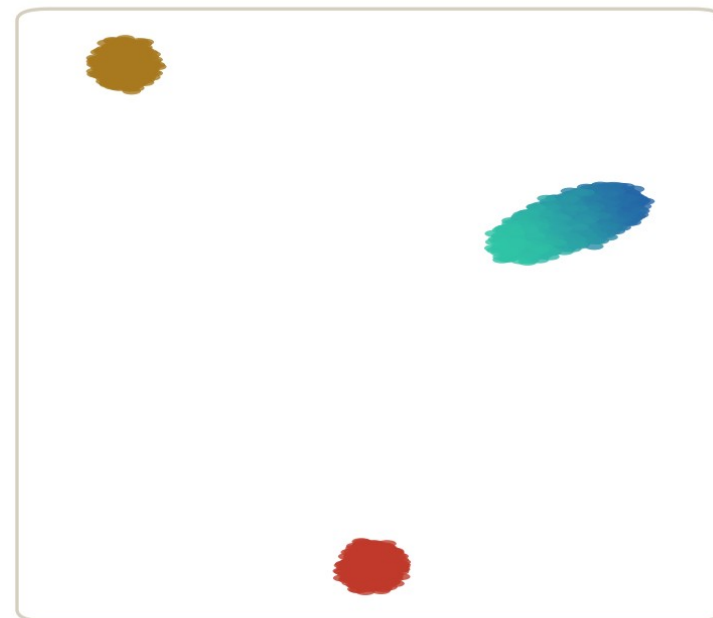
n.neighbors = 5



n.neighbors = 30 (default)



n.neighbors = 100



trusts 5 neighbours: local detail magnified

the local / global compromise

100 neighbours: global first, band squashed

one continuous band (blue-to-teal) + two discrete clusters — the knob moves the view, not the data

simulated data (run for real)

Small: local detail magnified. Large: global structure first, detail smoothed away. The default 30 is the compromise

Sweep min.dist: 0.01 / 0.3 / 0.8

```
for (m d in c(0.01, 0.3, 0.8)) {  
  tag <- gsub("\\.", "", as.character(m d)) # "001" "03" "08"  
  pbmc <- RunUMAP(pbmc, dims = 1:10, min.dist = m d,  
    reduction.name = paste0("umap_m d", tag),  
    reduction.key = paste0("UMAPm d", tag, "_"),  
    verbose = FALSE)  
}
```

min.dist sets how close points may sit in the embedding. Cosmetic only

min.dist: how tightly clusters pack

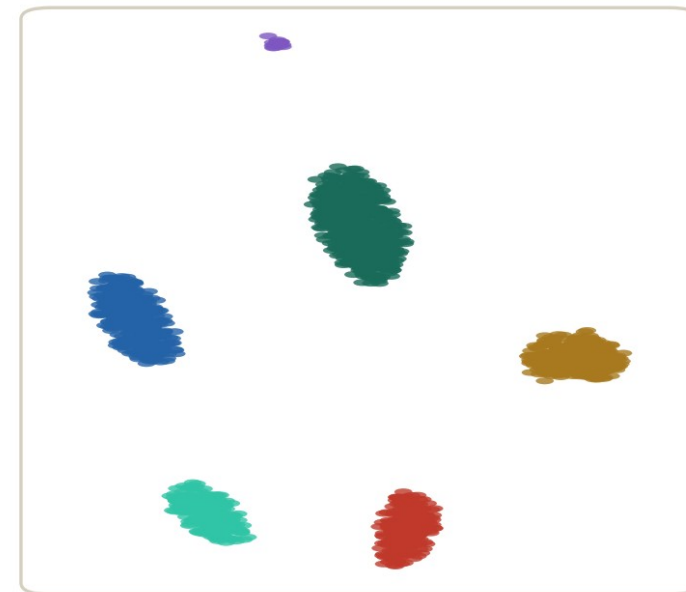
min.dist = 0.01



min.dist = 0.3 (default)



min.dist = 0.8



points may touch: tight clumps, looks 'super separated'

moderate packing

points pushed apart: fluffy, clusters drift together

min.dist only sets how tightly points may pack — it changes the look, never any analysis

simulated data (run for real)

Small: hard clumps and huge whitespace — 'super separated'. Large: fluffy clusters drifting together

Two knobs, two jobs

Remember this table — it also decodes other people's figures

n.neighbors

the local-global trade-off

- changes the KNN graph — a structural knob
- small favours local detail; large favours global layout
- the counterpart of t-SNE's perplexity

min.dist

packing tightness

- does not touch the KNN graph — purely cosmetic
- small gives tight clumps; large gives loose clouds
- the whitespace is its sculpture, not your data's

What this section did

Two knobs swept



six embeddings live in
the object for side-by-
side DimPlots

Verdict: defaults suffice



30 / 0.3 serves PBMC
3k; deviations need a
written reason

Looks are not evidence



parameter-sculpted
separation convinces
no one — see the traps

04

t-SNE: the elder is not retired

One comparison run, and you know when you will still meet it

RunTSNE in one line

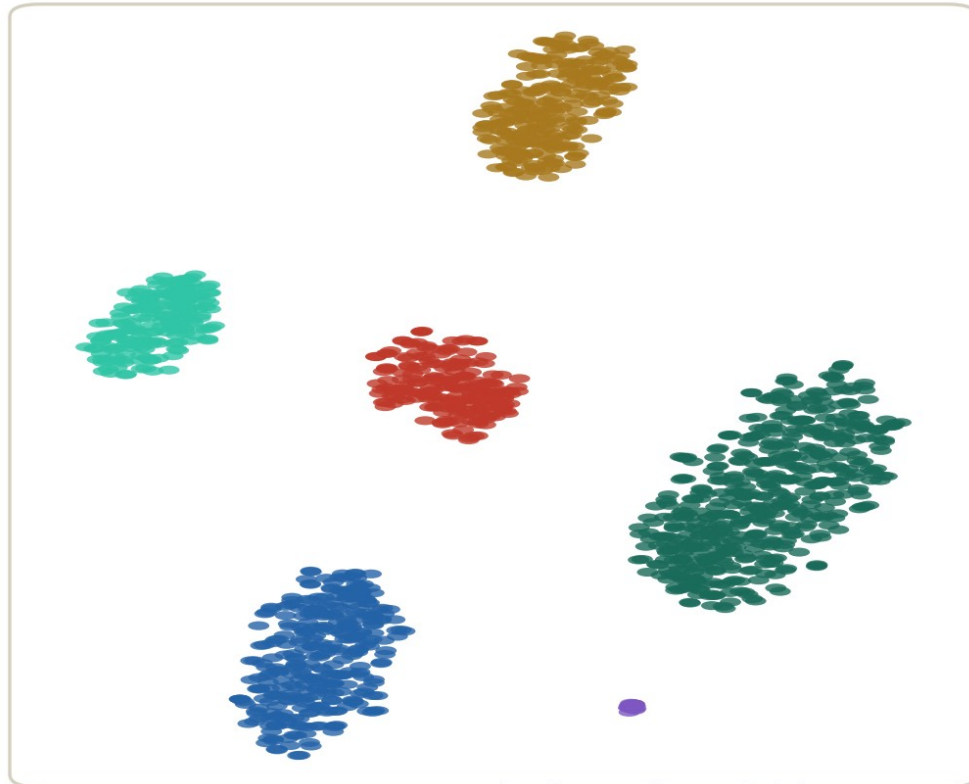
```
pbm c <- RunTSNE(pbm c, dims = 1:10) # perplexity 預設 30
```

```
DimPlot(pbm c, reduction = "tsne", label = TRUE) |  
DimPlot(pbm c, reduction = "umap", label = TRUE)
```

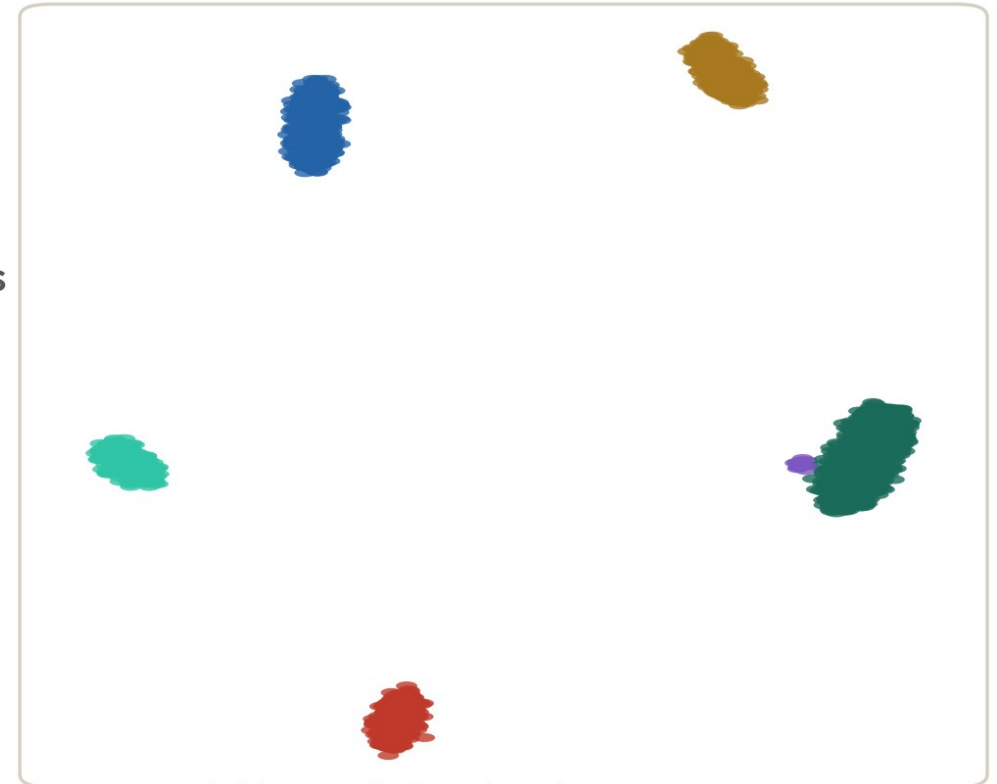
perplexity is to t-SNE what n.neighbors is to UMAP — the same 'how local' knob

t-SNE vs UMAP, same data, same colours

t-SNE (perplexity = 30)



UMAP (n.neighbors = 30)



same data
same colours

perplexity and n.neighbors are the same knob: how many neighbours define 'local'

simulated data (t-SNE actually run via sklearn) **between-cluster distances are unreadable in both**

t-SNE spreads whitespace evenly — its between-cluster distances mean even less than UMAP's

Which one, when

Default to UMAP; know when t-SNE still takes the stage

UMAP

this series' default

- fast, scales to large datasets
- keeps global structure somewhat better
- the ecosystem default in tutorials and atlases

t-SNE

you will still meet it twice

- reproducing or comparing with pre-2018 papers
- small datasets: local structure just as good
- its between-cluster distances deserve zero trust

05

A publication-grade DimPlot

Readable at a glance — including for colour-blind readers

label, repel, and a CB-safe palette

```
cb10 <- c("#0072B2", "#E69F00", "#009E73", "#CC79A7",  
          "#56B4E9", "#D55E00", "#F0E442", "#999999",  
          "#882255", "#44AA99") # Okabe-Ito 延伸 10 色
```

```
p.pub <- DimPlot(pbm.c, reduction = "umap",  
                label = TRUE, repel = TRUE, label.size = 4,  
                cols = cb10, pt.size = 0.3) +  
  NoLegend() + ggtitle(NULL)
```

Labels sit beside the clusters, repel stops them colliding; NoLegend kills the eye-hopping legend

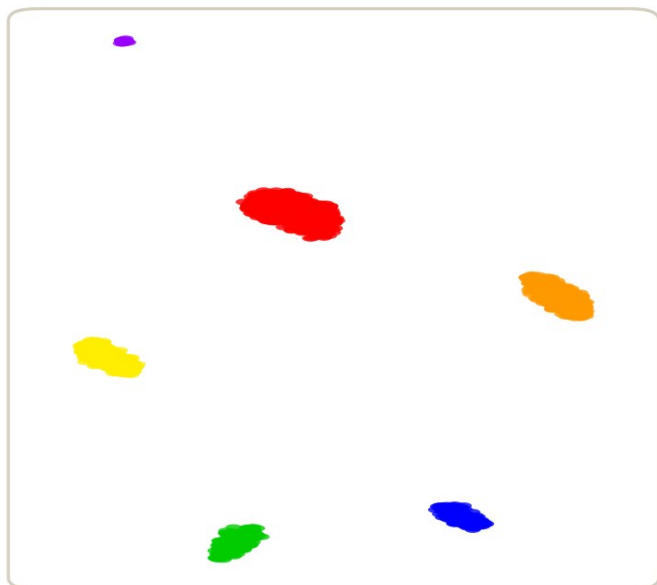
Export: size and font decided together

```
p.pub <- p.pub +  
  theme(axis.title = element_text(size = 9),  
        axis.text = element_text(size = 8))  
  
ggsave("output/B09_umap_final.pdf", p.pub,  
       width = 12, height = 10, units = "cm") # 半欄圖  
ggsave("output/B09_umap_final.png", p.pub,  
       width = 12, height = 10, units = "cm", dpi = 600)
```

Decide the printed size first, then back out the font: nothing under 8 pt after scaling

The disaster vs the publication version

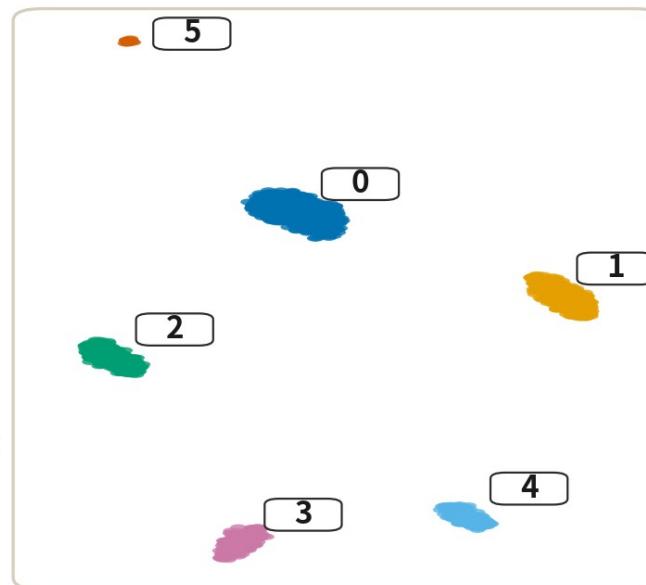
what the reviewer sees: the disaster



● cluster 0
● cluster 1
● cluster 2
● cluster 3
● cluster 4
● cluster 5

- ✗ red on green: CB readers lost
- ✗ IDs in a distant legend
- ✗ 6 pt text, invisible in print

what the reviewer sees: publication-grade



- ✓ Okabe-Ito CB-safe palette
- ✓ labels beside clusters (repel)
- ✓ exported text at 8 pt+

same result, two fates — publication-grade is not aesthetics, it is whether readers can parse it painlessly

And the caption states Seurat version, dims and seed — a figure ships with its parameters

06

Where people get hurt

Two traps — each demonstrated for real

Trap 1: computing statistics on UMAP coordinates

What it is

Using UMAP coordinates for distances, densities or trajectories, reported as quantitative results.



Why it happens

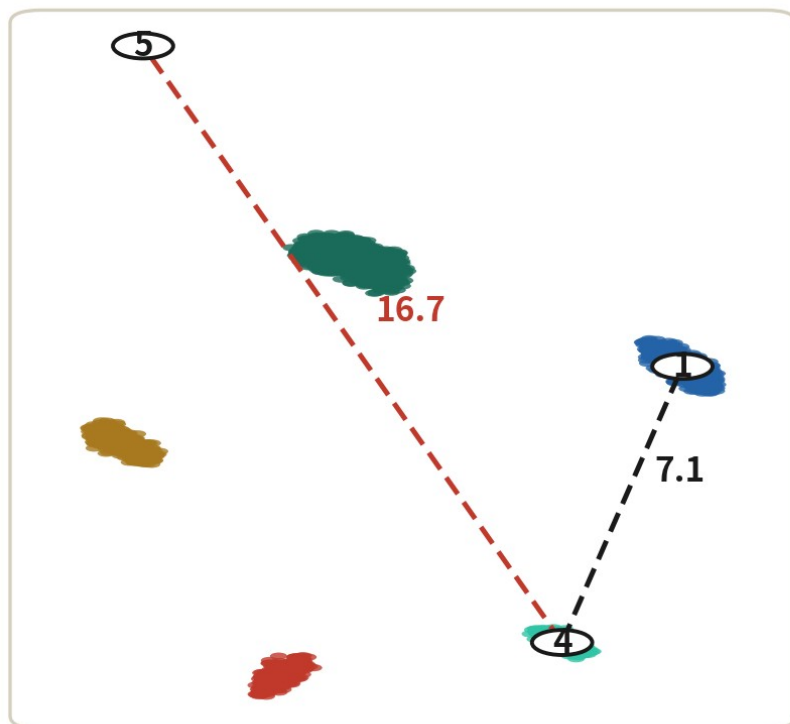
The coordinates are right there, `dist()` is one line away, and the picture looks like a real space.

What it costs you

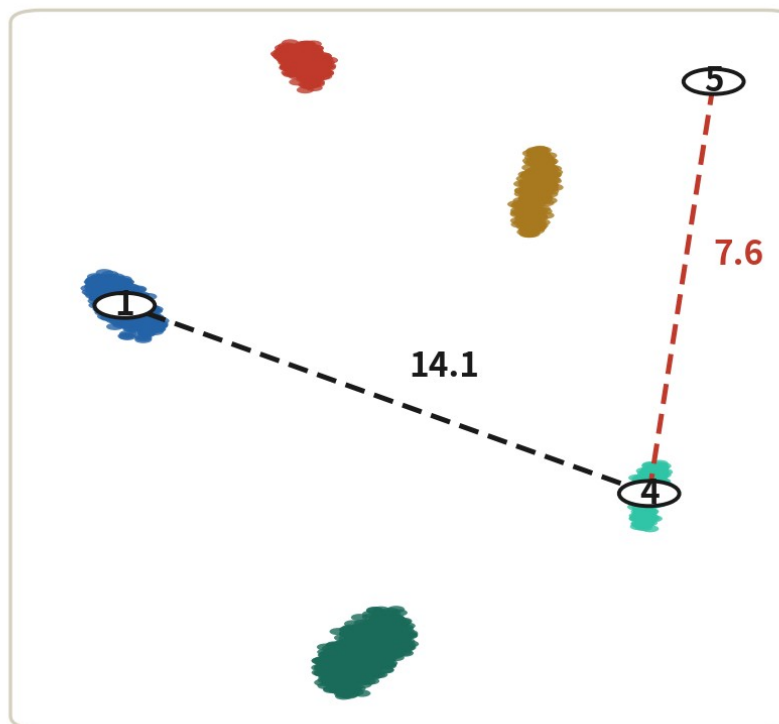
Change the seed and 'who is near whom' flips outright. Your statistic stands on sand.

Measured: distance rankings flip with the seed

seed A



seed B



same data

seed A:
4 closer to 1

seed B:
4 closer to 5

15 pairwise distances
rank correlation
 $\rho = -0.04$

'who is near whom' flips with the seed — statistics on UMAP distances stand on sand

simulated data (run for real)

Seed A: cluster 4 near 1. Seed B: cluster 4 near 5. Rank correlation across all 15 pairs: -0.04

Trap 2: tuning until the clusters separate

What it is

Two clusters touch, so you squeeze min.dist and n.neighbors until they 'separate cleanly' — and present that as evidence.



Why it happens

Same psychology as B8's made-to-order cluster count: the picture matches my hope, so it feels like proof.

What it costs you

Even a continuum with a 5% bridge can be squeezed into 'two crisp clusters'. That whitespace is sculpted by knobs, not biology.

Measured: whitespace made to order

n.neighbors=50, min.dist=0.8



'one continuum'

n.neighbors=5, min.dist=0.01



'two crisply separated clusters'

two end populations + a 5% bridge — the right panel hides the bridge inside the knobs

simulated data (run for real)

what this means

same data,
two conclusions

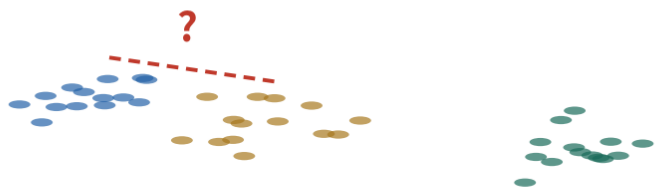
'they separate
on the plot'
is not evidence

evidence lives in
markers and
PC space

Same data: 'one continuum' on the left, 'two clusters' on the right. Evidence lives in markers and PC space, never in the gap

The three A5 misreads, one-page recap

misread 1: distances



✗ 'A sits near B, so they are similar'

✓ measure distance in PC space

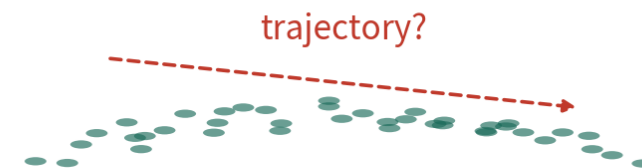
misread 2: areas



✗ 'big blob = big heterogeneity'

✓ area is sculpted by min.dist

misread 3: shapes



✗ 'elongated = a trajectory'

✓ shape is an optimization artifact

the three A5 warnings — now you can verify them yourself: change the seed or min.dist and watch

schematic

Distance, area, shape — none is readable. Today you own the tools to verify all three claims yourself

TRAPS, SUMMED UP

**Only one thing on a UMAP deserves trust:
which cells belong to which cluster.
Everything else is stage scenery.**

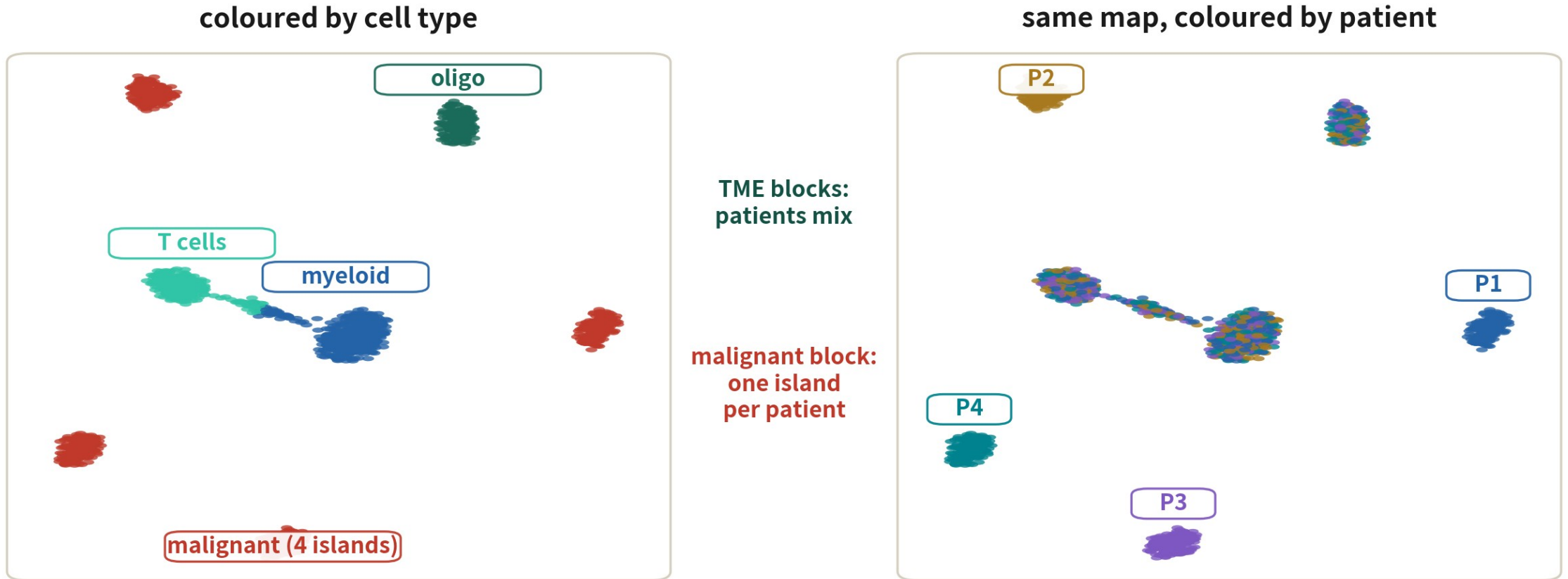
And even that membership list was not computed by UMAP — B8 settled it in PC space.



Strategy Room: Complex Samples

You can read a UMAP now — so imagine the map shows not PBMC, but a tumor

First, the four continents: TME connects, malignancy islands

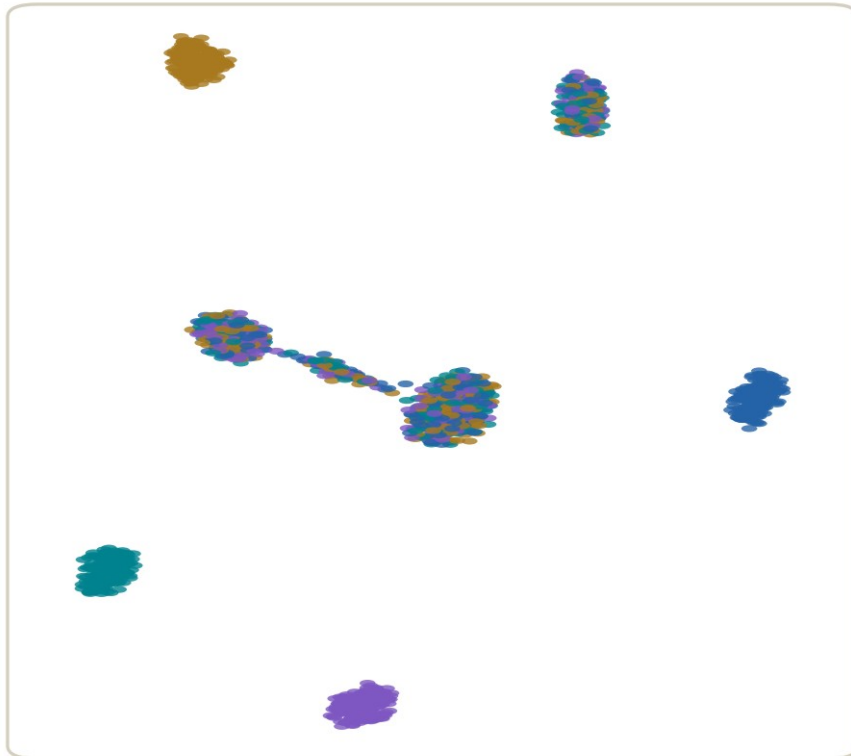


first glance at a tumor UMAP: find the four continents — the TME mixes across patients and connects; malignant cells island per patient
simulated data (run for real)

Myeloid, T and oligo mix across patients; malignant cells island per patient — each carries its own CNV combination

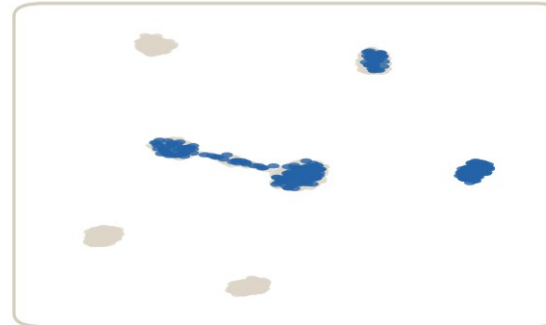
Per-patient panels: the standard tumor figure

everyone: coloured by patient



split.by = "patient": one panel per patient

P1 • 343 cells



P2 • 347 cells



P3 • 343 cells



P4 • 357 cells



the panels answer three questions: whose island is private? does every patient feed the TME? who has too few cells to conclude alone?

simulated data (run for real)

Just group.by plus split.by — this episode taught the tool; the difference is knowing to draw this set

The same misreads, twice as dangerous on the malignant continent

Distance and shape were already unreadable on PBMC; tumors stack patient identity and integration on top

PBMC

flips with a seed

- between-cluster distances: rank correlation ≈ 0
- shape: an optimization by-product
- yet the membership list stays trustworthy

Tumor

island distances mix in patient identity and integration

- two patients' islands sitting close $\neq$ similar subtypes
- four malignant clusters $\neq$ four subtypes — that is four patients; score states instead
- turn integration on and the islands can be pulled into one blob (B11)

Episode checklist



RunUMAP dims inherit B7's 1:10, with a reason



all three seed levels handled: set.seed, seed.use, single-thread + versions



n.neighbors and min.dist swept; you know what each controls



analysis stays in PC space; UMAP coords enter no statistics



you can recite the three A5 misreads: distance, area, shape



pub figure: CB-safe colours, on-plot labels, big fonts, params in caption

All six boxes ticked before you move to B10

ONE-LINE SUMMARY

**Analysis lives in PC space, pictures live on UMAP;
pin the seed and write the knobs down.**

Do these two things and your figure is both honest and reproducible.

Check yourself 1

Clustering, cell-cell distances, DE — which space do these analyses use?

- A The 2D UMAP coordinates
- B PC space (the first 10 PCs in this series)
- C The raw expression matrix, no reduction
- D t-SNE coordinates — more reliable than UMAP

Answer B. The KNN graph, distances and downstream statistics all live in PC space; UMAP and t-SNE are projections for your eyes, and their coordinates enter no computation.

Check yourself 2

Two UMAPs: tight clusters with big gaps (min.dist=0.01) vs fluffy clusters sitting close (min.dist=0.8). Is the first one 'better clustering'?

- A Yes — bigger gaps mean better separation
- B Yes, but check n.neighbors too
- C No — min.dist is cosmetic; not one cell changed cluster
- D No comparison possible — the seeds differ

Answer C. Clustering was settled in B8; UMAP only draws it. min.dist sculpts how tightly points pack — the whitespace is the parameter's artwork, not a quality metric.

Check yourself 3

A reviewer demands a reproducible figure. Which package of habits is complete?

- A `set.seed(1234)` at the top of the script — done
- B Fixed `seed.use` + version records (`renv/sessionInfo`) + uwot's default single-thread + a rerunnable script
- C Run it several times and archive the prettiest
- D Switch to t-SNE — it has no randomness

Answer B. A global `set.seed` cannot reach inside `RunUMAP`; you need the function-level `seed.use` plus implementation-level versions and threading, wrapped in a script that reruns end to end. A covers one level, C is denial, and D is wrong — t-SNE is just as random.

NEXT EPISODE

B10: annotation — markers, SingleR, Azimuth

Ten clusters on the map, currently named 0 through 9. Which is T? Which is monocyte?

Next: manual and automated annotation — and who to believe when they disagree.